

Using Propensity Scores

Lucy D'Agostino McGowan

Wake Forest University

Propensity scores

Matching

Weighting

Stratification

Direct Adjustment

...



Ingredients

150g unsalted butter, plus extra for greasing
150g plain chocolate, broken into pieces
150g plain flour
½ tsp baking powder
½ tsp bicarbonate of soda
200g light muscovado sugar
2 large eggs

Method

1. Heat the oven to 160C/140C fan/gas 3. Grease and base line a 1 litre heatproof glass pudding basin and a 450g loaf tin with baking parchment.
2. Put the butter and chocolate into a saucepan and melt over a low heat, stirring. When the chocolate has all melted remove from the heat.



estimand

estimator

estimate

Propensity scores

Matching

Weighting

Stratification

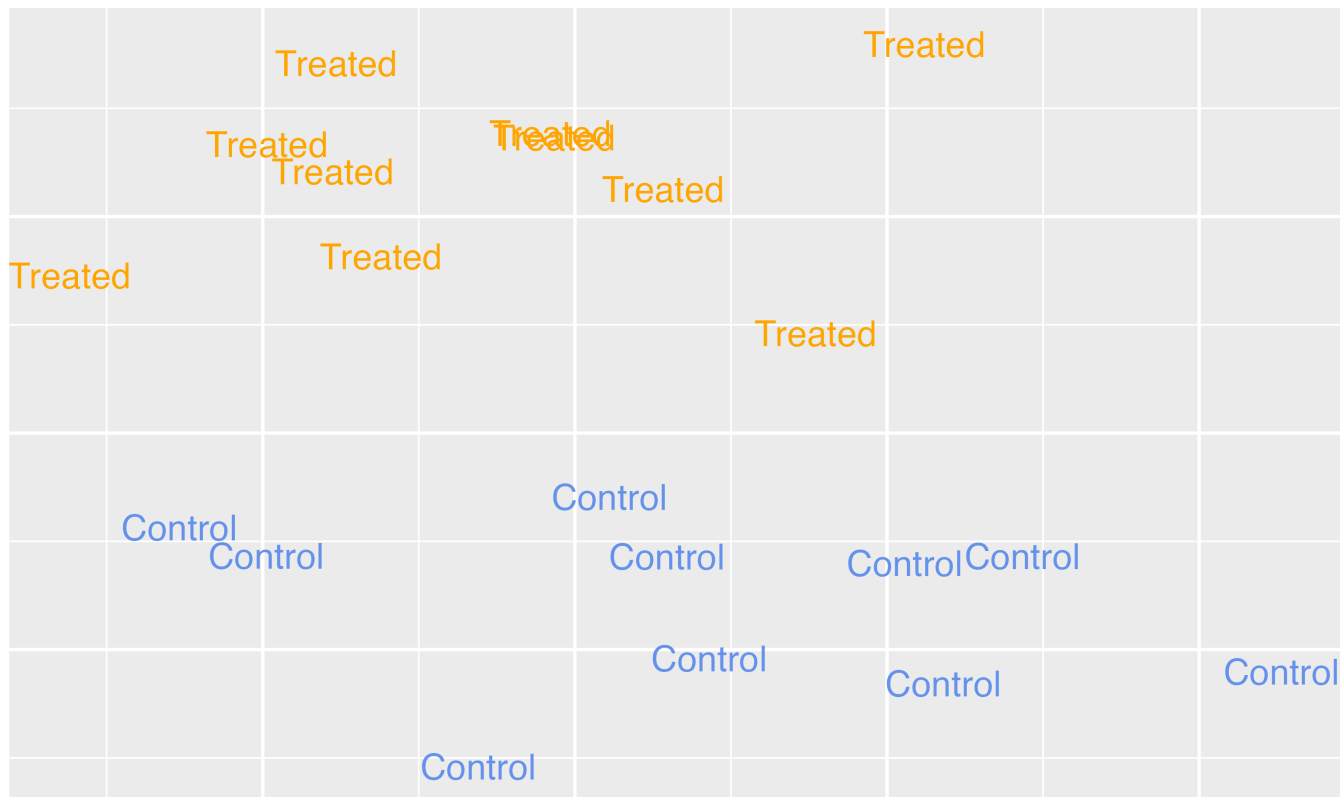
Direct Adjustment

...

Target estimands

Average Treatment Effect (ATE)

$$\tau = E[Y(1) - Y(0)]$$



Target estimands

Estimand	Target population	Example Research Question
ATE	Full population	<i>Should we decide whether to have extra magic hours all mornings to change the posted wait time for Seven Dwarfs Mine Train between 9-10 AM?</i> <i>Should a specific policy be applied to all eligible observations?</i>

Target estimands

Average Treatment Effect among the Treated (ATT)

$$\tau = E[Y(1) - Y(0) | Z = 1]$$

Target estimands

Estimand	Target population	Example Research Question
ATT	Exposed (treated) observations	<i>Should we stop extra magic hours to change the posted wait time for Seven Dwarfs Mine Train between 9-10 AM?</i> <i>Should we stop our marketing campaign to those currently receiving it?</i> <i>Should medical providers stop recommending treatment for those currently receiving it?</i>

Matching in R (ATT)

```
1 library(MatchIt)
2 m <- matchit(
3   net_num ~ income + health + temperature,
4   data = net_data
5 )
6 m
```

A `matchit` object

- method: 1:1 nearest neighbor matching without replacement
- distance: Propensity score
 - estimated with logistic regression
- number of obs.: 1752 (original), 908 (matched)
- target estimand: ATT
- covariates: income, health, temperature

Matching in R (ATT)

```
1 matched_data <- get_matches(m, id = "i")
2 as_tibble(matched_data)
```

```
# A tibble: 908 × 14
```

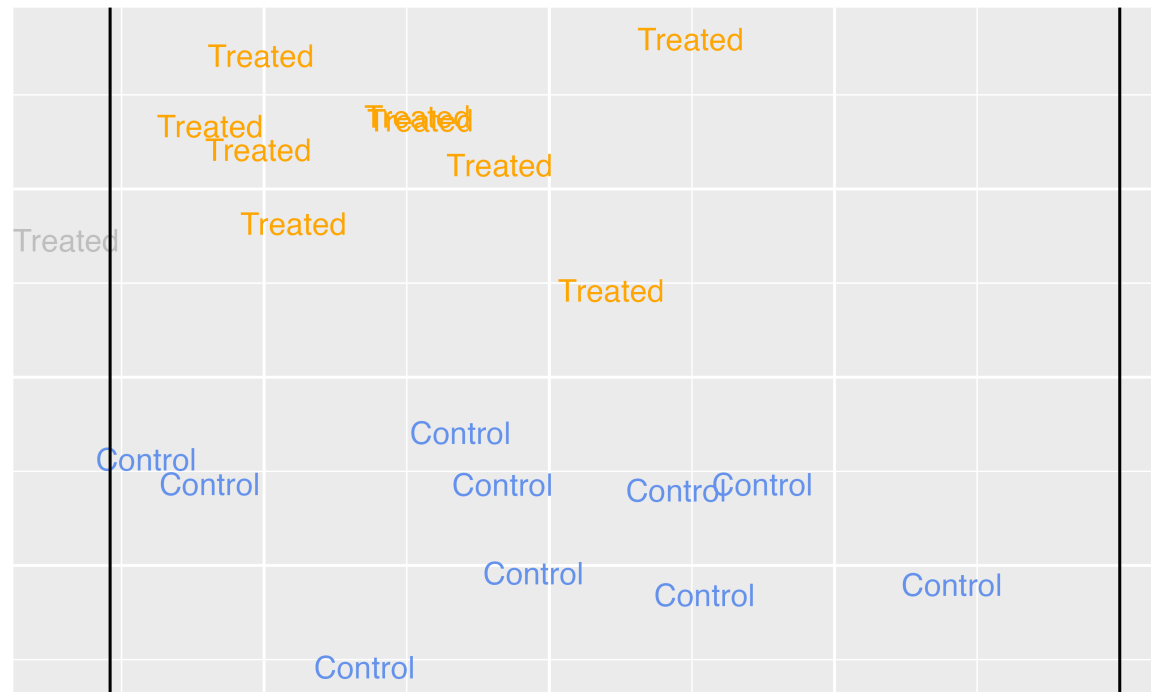
	i	subclass	weights	id	net	net_num	malaria_risk
	<chr>	<fct>	<dbl>	<int>	<lgl>	<int>	<dbl>
1	16	1	1	16	TRUE	1	15
2	109	1	1	109	FALSE	0	22
3	18	2	1	18	TRUE	1	28
4	1652	2	1	1652	FALSE	0	39
5	19	3	1	19	TRUE	1	18
6	64	3	1	64	FALSE	0	22
7	22	4	1	22	TRUE	1	17
8	329	4	1	329	FALSE	0	22
9	28	5	1	28	TRUE	1	22
10	706	5	1	706	FALSE	0	25

```
" - 800 - - - - -
```

Target estimands

Average Treatment Effect among the Unexposed / Controls (ATU / ATC)

$$\tau = E[Y(1) - Y(0) | Z = 0]$$



Target estimands

Estimand	Target population	Example Research Question
ATU	Unexposed (control) observations	<i>Should we add extra magic hours for all days to change the posted wait time for Seven Dwarfs Mine Train between 9-10 AM?</i> <i>Should we extend our marketing campaign to those not receiving it?</i> <i>Should medical providers extend treatment to those not currently receiving it?</i>

Matching in R (ATU)

```
1 m <- matchit(  
2   net_num ~ income + health + temperature,  
3   data = net_data,  
4   estimand = "ATC"  
5 )  
6 m
```

A `matchit` object

- method: 1:1 nearest neighbor matching without replacement
- distance: Propensity score
 - estimated with logistic regression
- number of obs.: 1752 (original), 908 (matched)
- target estimand: ATC
- covariates: income, health, temperature

`matchit()` spells this estimand **ATC**

Target estimands

Average Treatment Effect among the Matched (ATM)

Target estimands

Estimand	Target population	Example Research Question
ATM	Evenly matchable	<i>Are there some days we should change whether we are offering extra magic hours in order to change the posted wait time for Seven Dwarfs Mine Train between 9-10 AM?</i> <i>Is there an effect of the exposure for some observations?</i> <i>Should those at clinical equipoise receive treatment?</i>

Matching in R (ATM)

```
1 m <- matchit(  
2   net_num ~ income + health + temperature,  
3   data = net_data,  
4   link = "linear.logit",  
5   caliper = 0.1  
6 )  
7 m
```

The caliper is the largest difference between two observations we accept as a match, measured in standard deviations of the propensity score (here, on the linear logit scale). Observations that cannot be matched within it are discarded

Matching in R (ATM)

A `'matchit'` object

- method: 1:1 nearest neighbor matching without replacement
- distance: Propensity score [caliper]
 - estimated with logistic regression and linearized
- caliper: <distance> (0.036)
- number of obs.: 1752 (original), 898 (matched)
- target estimand: ATT
- covariates: income, health, temperature

Matching in R (ATM)

```
1 matched_data <- get_matches(m, id = "i")
2 as_tibble(matched_data)
```

```
# A tibble: 898 × 14
```

	i	subclass	weights	id	net	net_num	malaria_risk
	<chr>	<fct>	<dbl>	<int>	<lgl>	<int>	<dbl>
1	16	1	1	16	TRUE	1	15
2	109	1	1	109	FALSE	0	22
3	18	2	1	18	TRUE	1	28
4	1652	2	1	1652	FALSE	0	39
5	19	3	1	19	TRUE	1	18
6	657	3	1	657	FALSE	0	22
7	22	4	1	22	TRUE	1	17
8	329	4	1	329	FALSE	0	22
9	28	5	1	28	TRUE	1	22
10	706	5	1	706	FALSE	0	25

"

Your Turn 1

Using the propensity scores you created in the previous exercise, create a “matched” data set using the ATM method with a caliper of 0.2.

Propensity scores

Matching

Weighting

Stratification

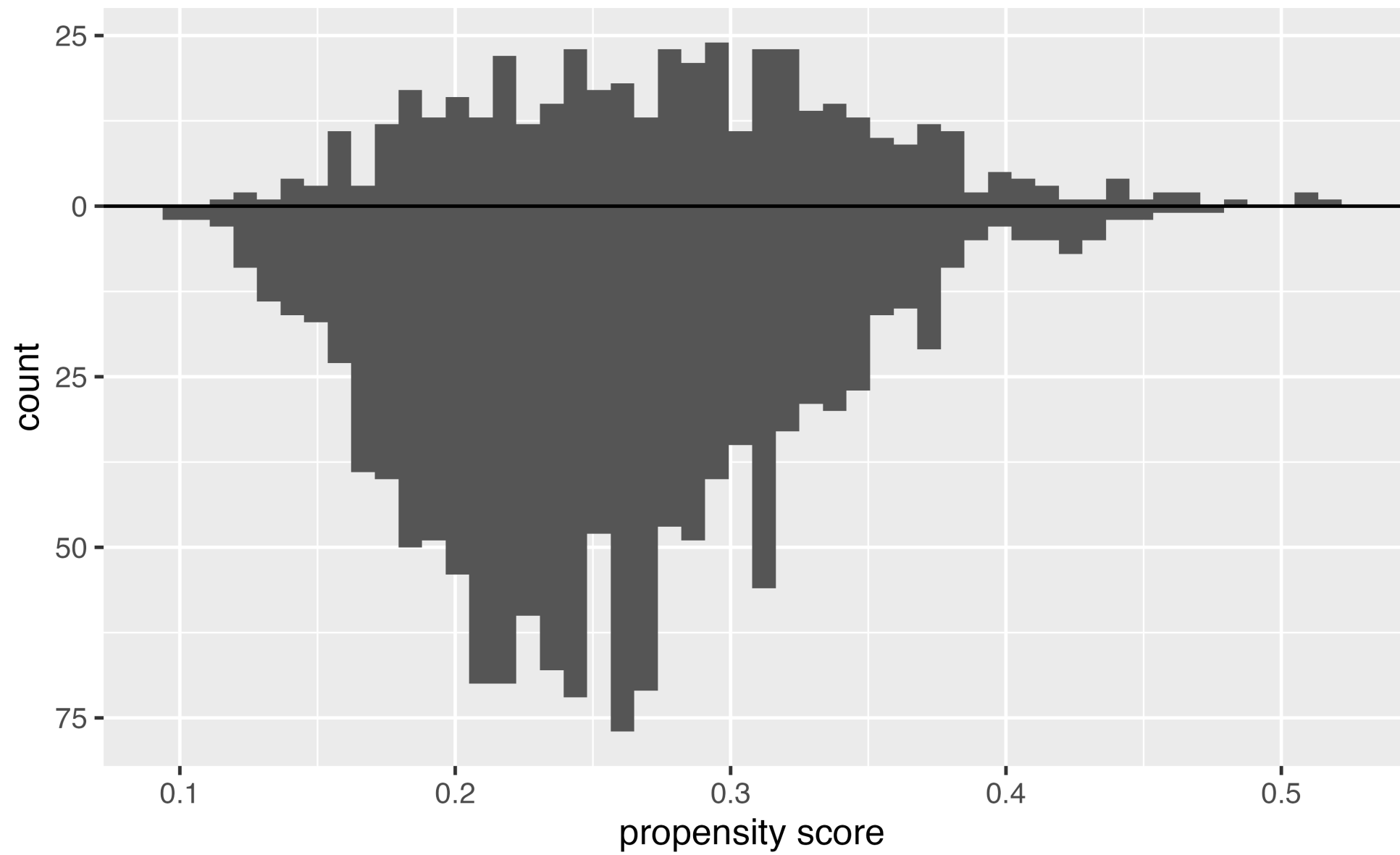
Direct Adjustment

...

Matching or weighting?

- Weighting is statistically more efficient; we recommend it when possible
- Matching has a distinct advantage: it's easy to understand

Histogram of propensity scores



Target estimands: ATE

Average Treatment Effect (ATE)

$$w_{ATE} = \frac{Z_i}{p_i} + \frac{1 - Z_i}{1 - p_i}$$

$$1 \cdot (Z / p) + ((1 - Z) / (1 - p))$$

Tilting functions

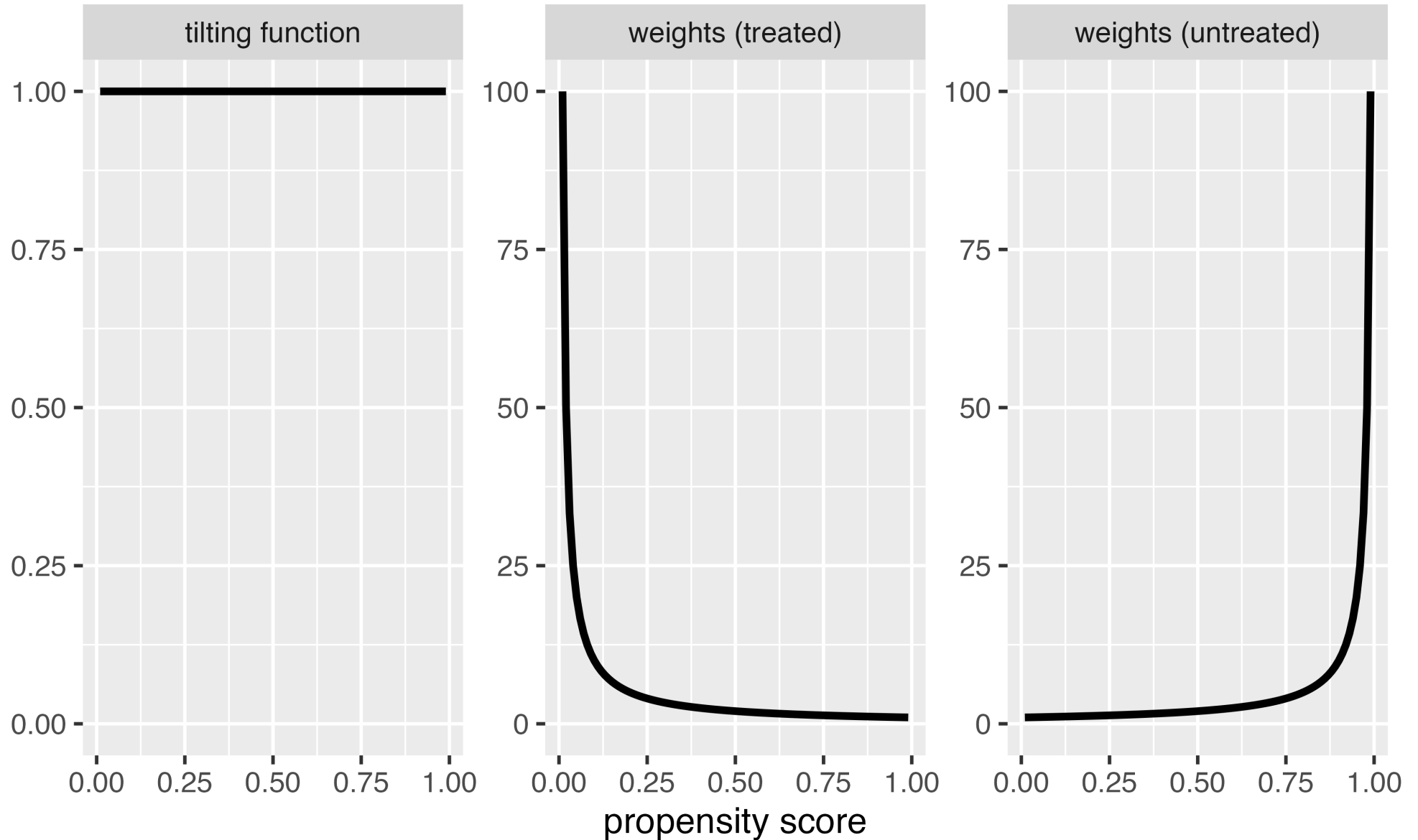
Every weight is the ATE weight times a **tilting function**, $h(p_i)$

$$w_i = \frac{h(p_i)Z_i}{p_i} + \frac{h(p_i)(1 - Z_i)}{1 - p_i}$$

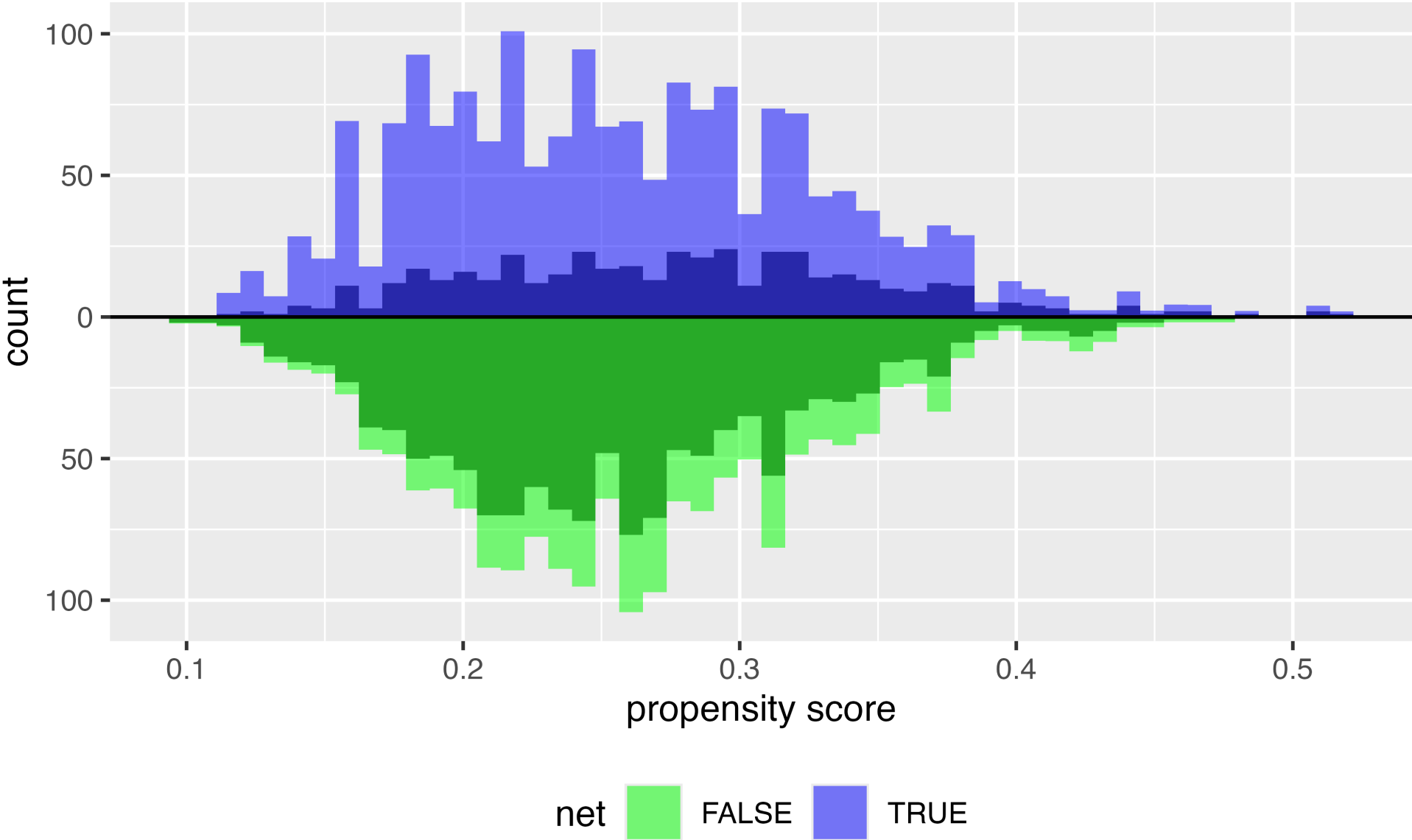
The tilting function reshapes the population we are targeting

For the ATE, the tilting function is $h(p_i) = 1$: the population is unchanged and the weights are unbounded

ATE tilting function



ATE



Target estimands: ATT & ATU

Average Treatment Effect Among the Treated (ATT)

$$w_{ATT} = \frac{p_i Z_i}{p_i} + \frac{p_i(1 - Z_i)}{1 - p_i}$$

$$1 \quad ((p * Z) / p) + ((p * (1 - Z)) / (1 - p))$$

The tilting function is $h(p_i) = p_i$: the treated get a weight of 1

Target estimands: ATT & ATU

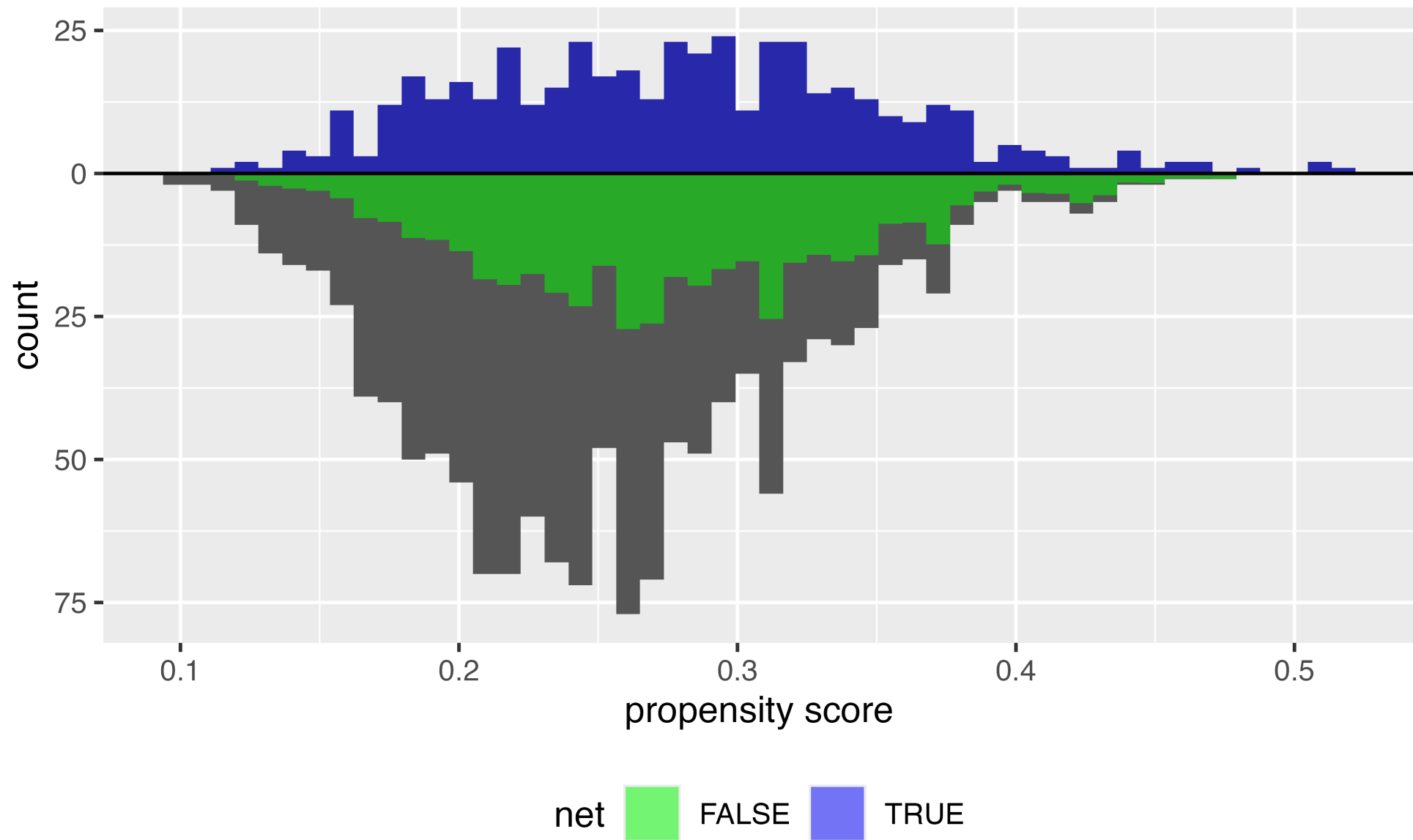
Average Treatment Effect Among the Unexposed (ATU)

$$w_{ATU} = \frac{(1 - p_i)Z_i}{p_i} + \frac{(1 - p_i)(1 - Z_i)}{(1 - p_i)}$$

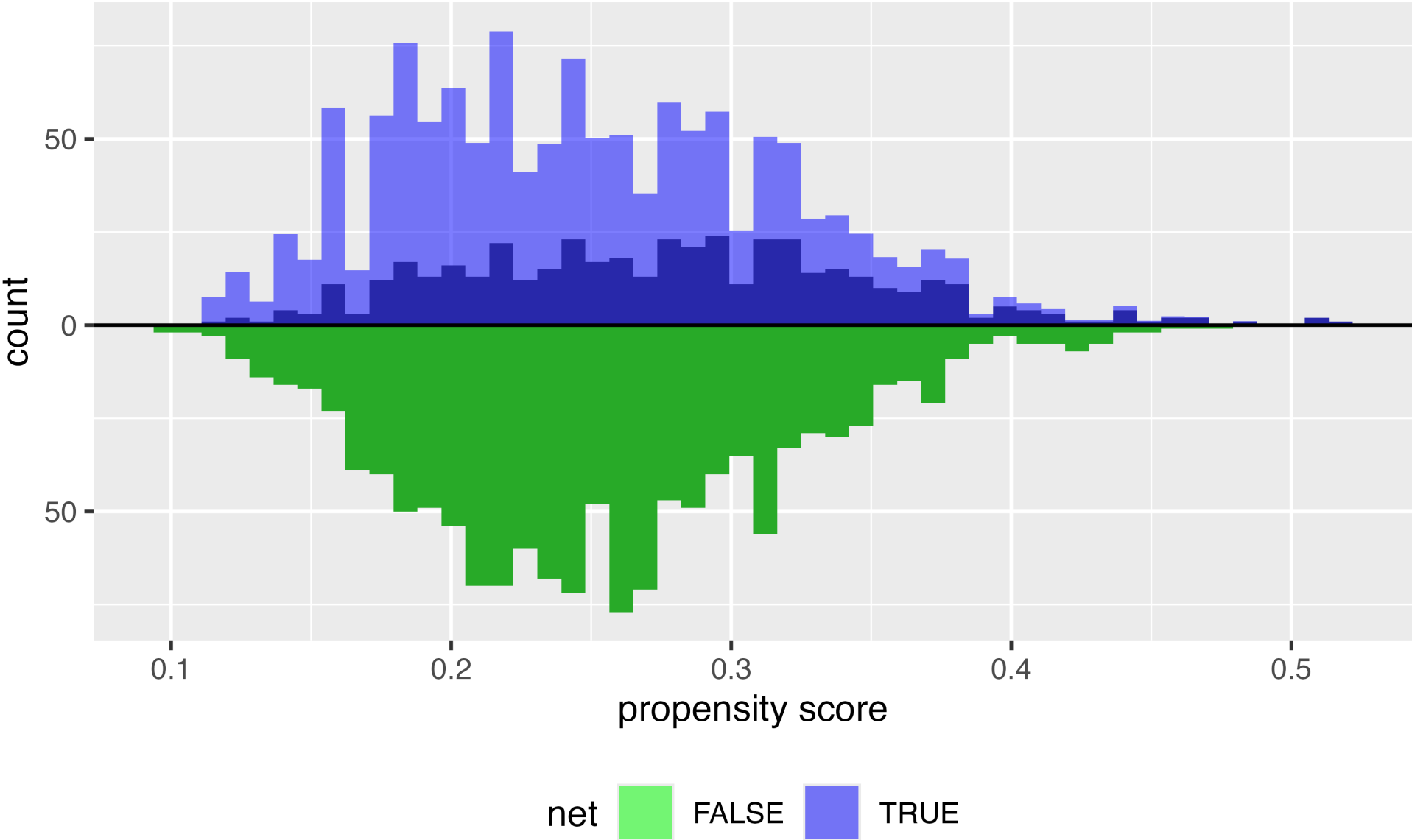
$$1 \left(\frac{((1 - p) * Z)}{p} + \frac{((1 - p) * (1 - Z))}{(1 - p)} \right)$$

The tilting function is $h(p_i) = 1 - p_i$: the untreated get a weight of 1

ATT



ATU



Target estimands: ATM & ATO

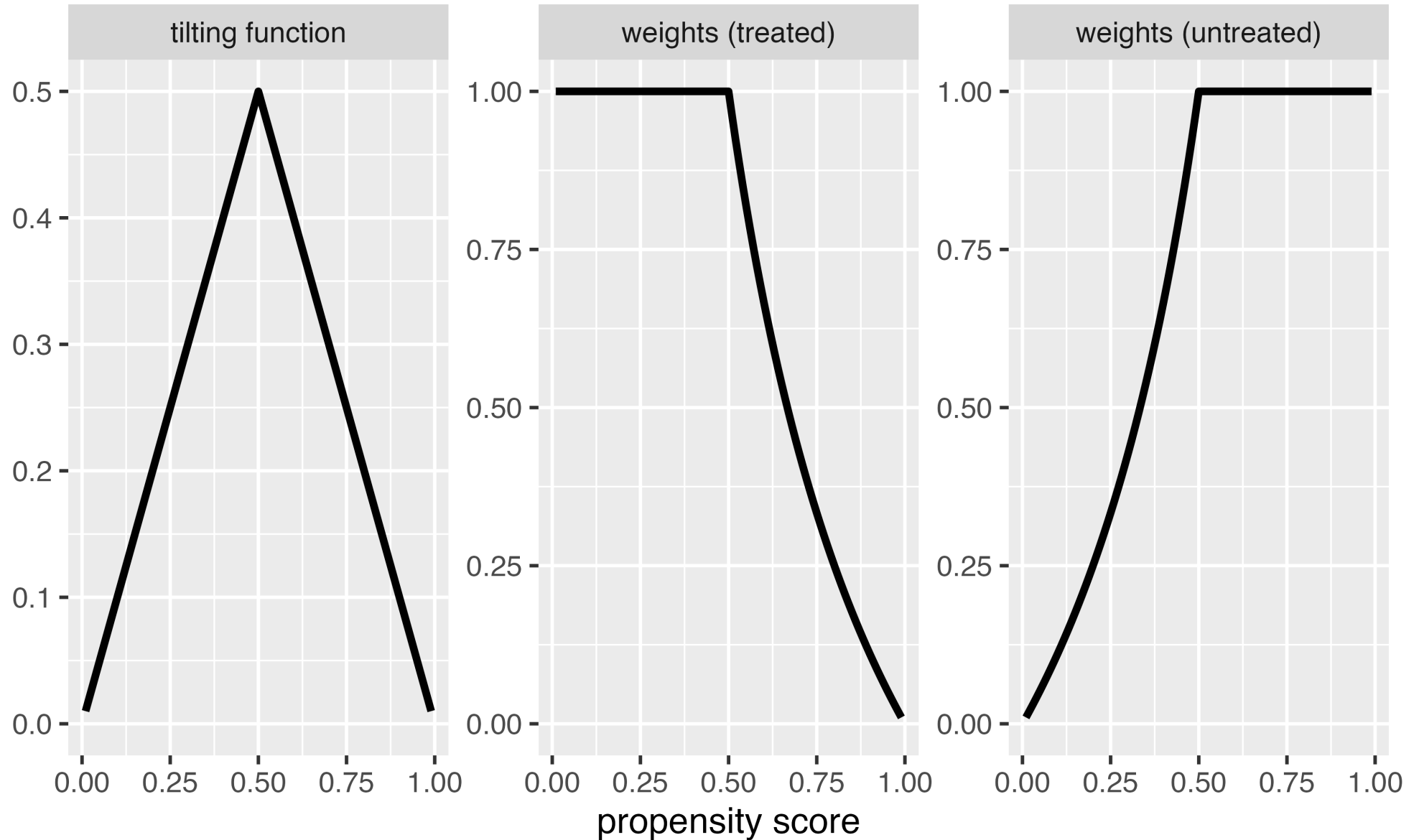
Average Treatment Effect Among the Evenly Matchable (ATM)

$$w_{ATM} = \frac{\min\{p_i, 1 - p_i\}}{Z_i p_i + (1 - Z_i)(1 - p_i)}$$

```
1 pmin(p, 1 - p) / (Z * p + (1 - Z) * (1 - p))
```

The tilting function is $h(p_i) = \min\{p_i, 1 - p_i\}$: a triangle peaking at 0.5

ATM tilting function



Target estimands: ATM & ATO

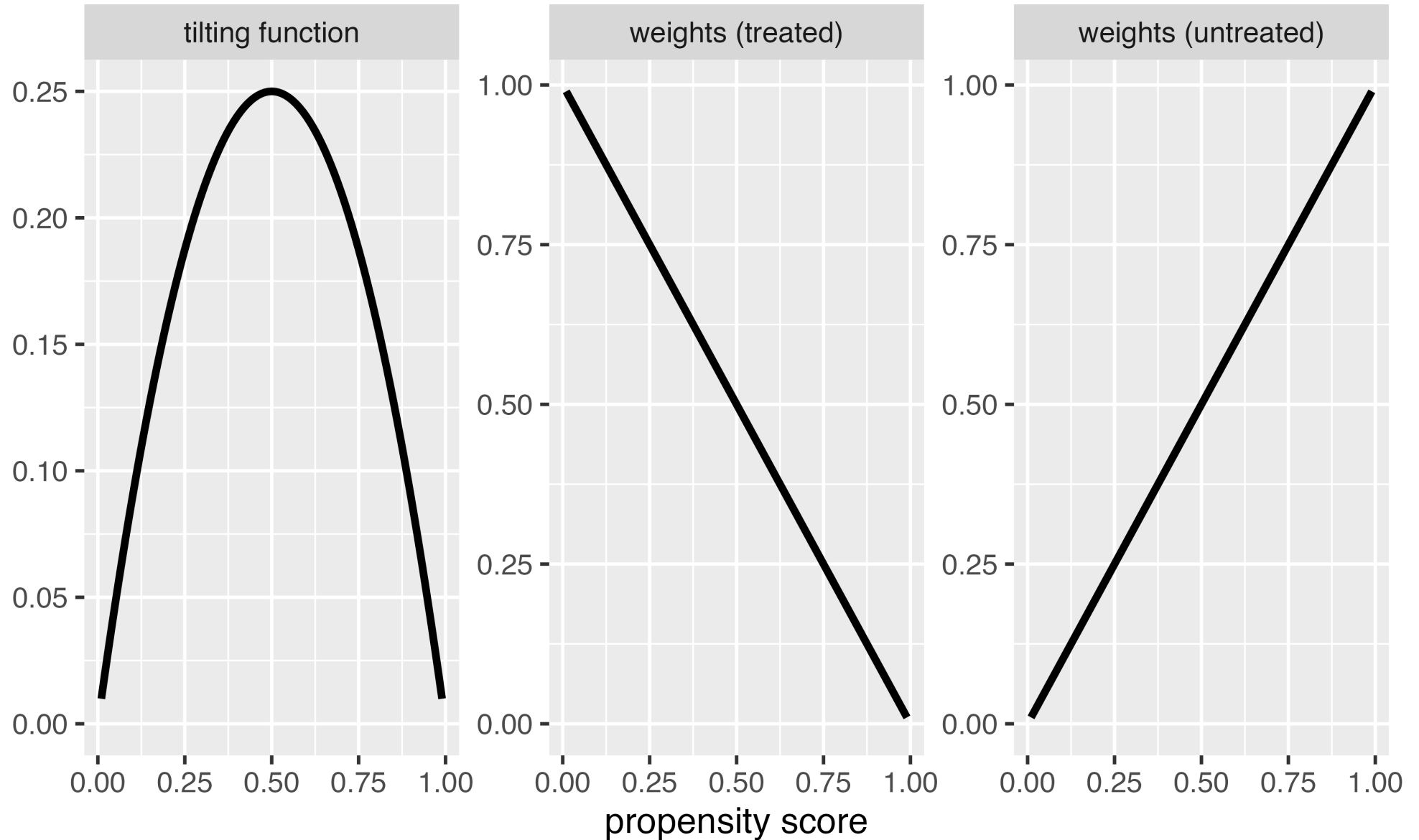
Average Treatment Effect Among the Overlap Population

$$w_{ATO} = (1 - p_i)Z_i + p_i(1 - Z_i)$$

$$1 \quad (1 - p) * Z + p * (1 - Z)$$

The tilting function is $h(p_i) = p_i(1 - p_i)$: smooth, peaking at 0.5, and the weights are bounded by 0 and 1

ATO tilting function

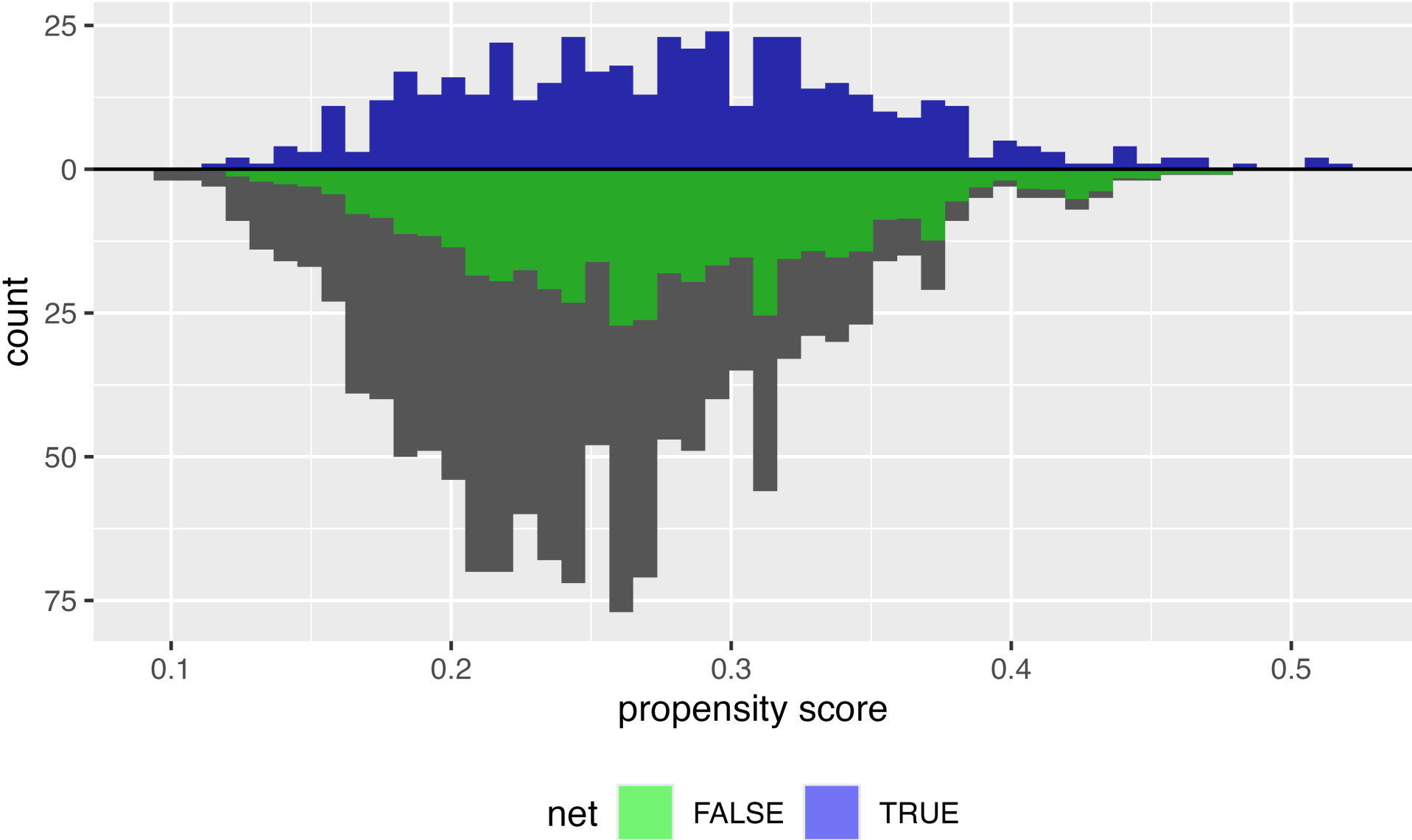


Target estimands

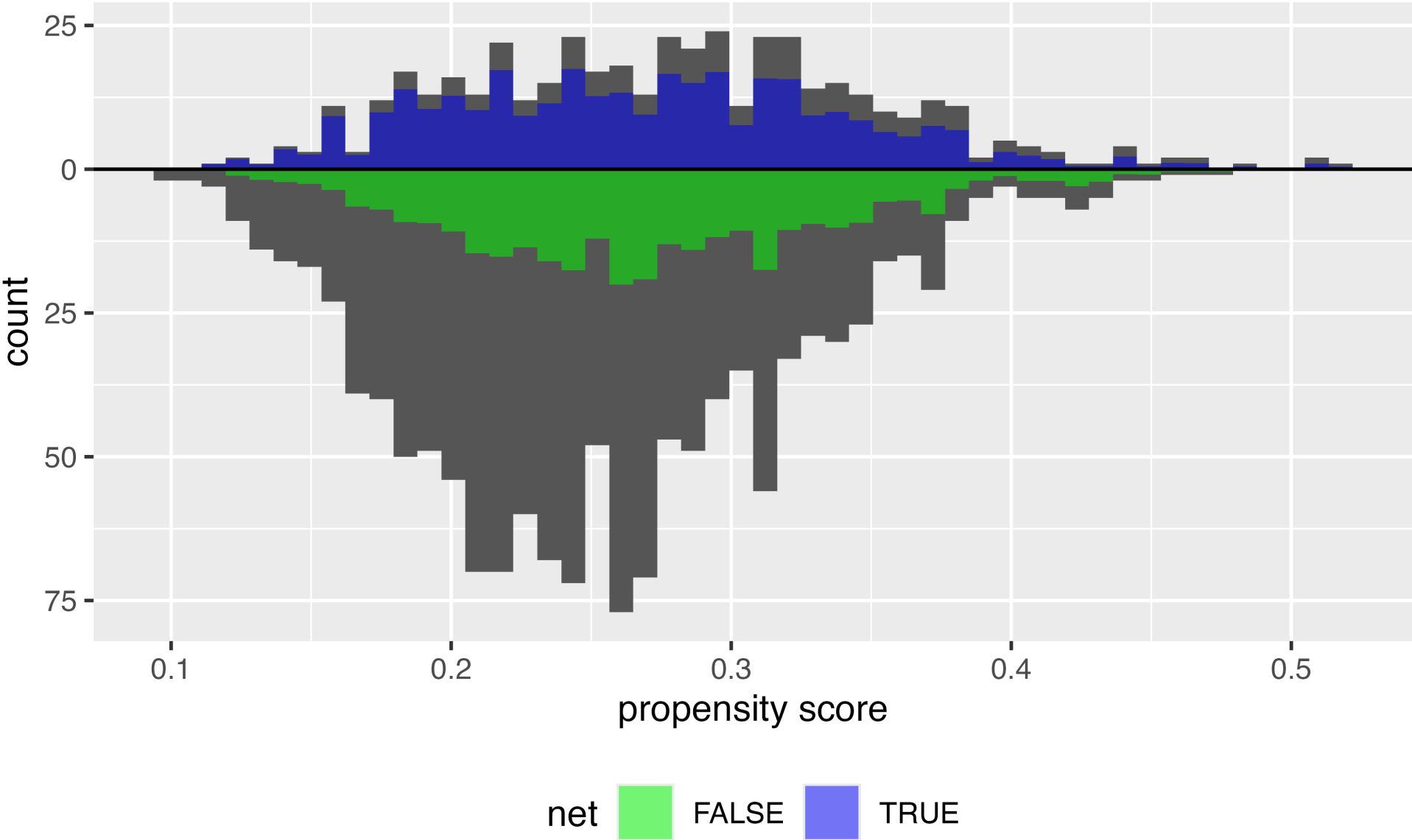
Estimand	Target population	Example Research Question
ATO	Overlap population	<i>Same as ATM</i>

We will compare these populations with the effective sample size when we check balance

ATM



ATO



ATE in R



Average Treatment Effect (ATE) $w_{ATE} = \frac{Z_i}{p_i} + \frac{1-Z_i}{1-p_i}$

```
1 library(propensity)
2 df <- propensity_model |>
3   augment(type.predict = "response", data = net_data) |>
4   mutate(w_ate = wt_ate(.fitted, net))
```

Extreme weights

- The minimum of the ATE weights is 1, but the maximum is unbounded
- The closer the probability of the observed exposure is to 0, the higher the weight
- **Extreme weights *destabilize*** our estimate, resulting in worse precision and potentially worse bias

Extreme weights

```
1 df |>
2   arrange(desc(w_ate)) |>
3   select(net, income, health, temperature, .fitted, w_ate)
```


Extreme weights

```
# A tibble: 1,752 × 6
```

	net	income	health	temperature	.fitted	w_ate
	<lgl>	<dbl>	<dbl>	<dbl>	<dbl>	<psw{ate}>
1	TRUE	501	10	29.9	0.117	8.556008
2	TRUE	456	7	26.4	0.120	8.328450
3	TRUE	544	10	29.7	0.126	7.915342
4	TRUE	424	13	20.2	0.136	7.361471
5	TRUE	613	43	30.4	0.138	7.270849
6	TRUE	644	33	31.9	0.139	7.190567
7	TRUE	532	26	24.6	0.143	7.011099
8	TRUE	502	21	22.7	0.143	6.968780
9	TRUE	633	35	29.7	0.145	6.883178
10	TRUE	547	18	24.7	0.146	6.845732

```
"# A tibble: 1,752 × 6"
```

Stabilized weights

```
1 df <- df |>
2   mutate(
3     sw_ate = wt_ate(.fitted, net, stabilize = TRUE)
4   )
5
6 df |>
7   summarize(mean = mean(sw_ate), min = min(sw_ate), max = max(sw_ate))
```

```
# A tibble: 1 × 3
  mean    min    max
<dbl> <dbl> <dbl>
1  1.00 0.501  2.22
```

Stabilizing puts the marginal probability of the observed exposure in the numerator: the proportion exposed for exposed units, the proportion unexposed for unexposed units

Trimming and truncation

- **Trimming:** set an acceptable range of the propensity score and drop the observations outside it
- **Truncation:** set the values outside the range to the minimum or maximum of the range
- Some authors use these terms interchangeably, or with the opposite meanings

Trimming and truncation in R

```
1 df <- df |>
2   mutate(
3     trimmed_ps = ps_trim(.fitted, method = "adaptive") |>
4       ps_refit(propensity_model),
5     trimmed = is_unit_trimmed(trimmed_ps),
6     trunc_ps = ps_trunc(.fitted, method = "pctl", lower = 0.01, upper = 1)
7   )
8
9 df |>
10  select(net, income, health, temperature, .fitted, trimmed) |>
11  filter(trimmed)
```

Trimming and truncation in R

```
# A tibble: 2 × 6
  net      income health temperature .fitted trimmed
<lgl>   <dbl>   <dbl>         <dbl>    <dbl> <lgl>
1 FALSE    340     15          26.3  0.0987 TRUE
2 FALSE    351     21          27.1  0.0980 TRUE
```

Trimming and truncation

- Trimming and truncation, like using a caliper, change the population we are making inferences about
- The tilting function is 0 for the observations we trimmed and 1 for the ones that remain
- You can think of the resulting estimand as another type of overlap population

Assessing positivity

- Trimming and truncation respond to extreme propensity scores without saying where they came from
- **positively** searches the covariates for the region responsible

Propensity scores with a violation

```
1 pos_model <- glm(  
2   exposure ~ x1 + x2 + region,  
3   family = binomial(),  
4   data = pos_violations  
5 )  
6  
7 pos_df <- pos_model |>  
8   augment(type.predict = "response", data = pos_violations) |>  
9   mutate(w_ate = wt_ate(.fitted, exposure))  
10  
11 pos_df |>  
12   summarize(min = min(.fitted), max = max(.fitted), max_wt = max(w_ate))
```

```
# A tibble: 1 × 3  
      min      max max_wt  
  <dbl> <dbl>   <dbl>  
1 0.000107 1.000    473.
```


Which units did trimming drop?

```
1 pos_df |>
2   mutate(trimmed = is_unit_trimmed(ps_trim(.fitted, method = "adaptive"))) |>
3   group_by(structural = region == "b" & x2 > 1) |>
4   summarize(n = n(), trimmed = sum(trimmed))
```

```
# A tibble: 2 × 3
```

	structural	n	trimmed
	<lgl>	<int>	<int>
1	FALSE	924	396
2	TRUE	76	38

Trimming dropped half of the 76 units that can never be exposed, and 396 units that can

Locating the region directly

```
1 check_port(pos_violations, exposure, c(x1, x2, region)) |>
2   tidy() |>
3   filter(low_support) |>
4   select(description, n, prevalence)
```

```
# A tibble: 3 × 3
```

	description	n	prevalence
	<chr>	<int>	<dbl>
1	x1<-0.5027	305	0.0459
2	x1>=0.9626 & x1<1.154	53	0.962
3	x2>=1.063 & region=b	69	0

Your Turn 2

Using the propensity scores you created in the previous exercise, add the ATE weights to your data frame

Stretch: Using the same propensity scores, create ATM weights, plot the weighted distributions, and look for low-support regions with `check_port()`